

CSCE 637 COMPLEXITY THEORY · MODULE 02 · LECTURE NOTES

P, and What Tractable Means

The class, and the thesis behind it.

MODULE OUTCOMES

- Define P and state the Cobham–Edmonds thesis.
- Explain the arguments for and against the thesis.
- Give examples of problems whose membership in P was hard won.
- Explain closure properties of P and why they matter.
- State honestly what P does and does not capture.

1 The class, and the thesis

P IS THE PROBLEMS DECIDABLE IN POLYNOMIAL TIME, AND THE CHOICE IS DEFENDED RATHER THAN OBVIOUS

P is the class of decision problems solvable by a deterministic Turing machine in time $O(n^k)$ for some fixed k , where n is the input length in bits (Module 01 §2).

The Cobham–Edmonds thesis claims that **P captures the informal notion of 'tractable'** — and like the Church–Turing thesis it is a claim about a definition's adequacy (CSCE 627 Module 01 §2) rather than a theorem, so it is argued for rather than proved.

The case for it: **polynomials are closed under addition, multiplication, and composition**, so P is robust to the choice of machine model (Module 01 §1) and to composing algorithms (§3) — and no other natural time bound has both properties.

And the case against it is immediate and correct: n^{100} is not tractable by any standard, and 1.0001^n is perfectly fine for n up to a million. Both objections hold, and neither has produced a better definition in fifty years — which is the honest state of the matter rather than a defence.

The objection	The response
n^{100} is not tractable, so P is too generous.	True — and in practice the natural problems known to be in P have small exponents, mostly 1 to 4. The high-degree cases are artificial constructions (and §2's callout gives a reason why).
1.0001^n is fine for realistic n , so P is too restrictive.	True — and such algorithms are rare. Real exponential algorithms arising from search have base 2 or more, where the asymptotic and practical pictures agree.

The objection	The response
Constants are ignored entirely.	True — and they are model-dependent, so including them would destroy the robustness that makes the class meaningful (Module 01 §1). The omission is load-bearing.
It is a worst-case notion.	True — and Module 12 addresses it directly; the average-case alternatives require distributional assumptions nobody can justify (Module 01 §3).
It says nothing about parallelism or memory.	True — which is why NC and L exist as separate classes (Module 06), and why P is one measurement rather than the measurement.

Every objection is correct, and the class survives because no proposed replacement is both robust and closer to practice — which is a weaker defence than people generally assume, and it is the honest one. Being candid about this is more useful than overselling the thesis, because it sets the right expectation for what a P-membership result tells you (§4).

2 Membership results that were hard won

- **Linear programming.** Khachiyan's ellipsoid method, 1979, was the first polynomial-time algorithm — and it is unusable in practice, while the practical interior-point methods came later and the practical simplex method is exponential in the worst case (CSCE 669 Modules 03 and 07). **A clean case of the theory and the practice settling the same question differently.**
- **Primality.** AKS, 2002 — and the practical tests remain randomised (Module 08), so the polynomial algorithm settled a long-standing theoretical question and changed nothing operationally. **Which is worth knowing as an example of what a complexity result is and is not for.**
- **Maximum matching in general graphs.** Edmonds, 1965 — and that paper is where the polynomial-time thesis was first argued explicitly (see the callout).
- **Graph isomorphism.** Not known to be in P, not known to be NP-complete, and Babai's 2015 quasi-polynomial algorithm is the state of the art — faster than any known exponential bound and slower than any polynomial. **Probably in neither class.**
- **And integer factoring,** which is in neither P nor known to be NP-complete, **and on which most of deployed public-key cryptography rests** (CSCE 711's subject). **The last two matter structurally: NP-intermediate problems are believed to exist,** and these are the standing candidates — **so NP is not a two-tier structure of easy and complete, which Module 05 develops.**

EDMONDS' 1965 ARGUMENT IS THE ORIGINAL STATEMENT

Edmonds, presenting his matching algorithm, distinguished 'good' algorithms from exponential search and argued explicitly that the distinction was the right one to draw — six years before Cook's theorem gave the field its central question.

His argument was that an exponential algorithm expresses no insight: it is organised enumeration of possibilities, which is always available and tells you nothing. **Finding a polynomial algorithm requires understanding something about the problem's structure.**

Which is a better defence of the thesis than the robustness argument of §1, because it says what a polynomial algorithm *means* rather than only that the class is mathematically well behaved.

And it predicts the empirical pattern: natural problems in P have small exponents because the insight that puts them there is usually a strong structural one — a matroid (CSCE 669 Module 11 §2), a flow formulation, a total unimodularity — **and strong structure gives fast algorithms, not merely polynomial ones.** Which is §1's first response, with a reason attached rather than an observation.

3 Closure, and why it matters

P IS CLOSED UNDER THE OPERATIONS YOU ACTUALLY USE

Closed under complement — run the algorithm and swap the answer. **Which NP is *not* known to be** (Module 05 §1), **and that asymmetry is the entire content of co-NP** and one of the most useful structural facts in the course.

Closed under union, intersection, and concatenation, and most importantly **under polynomial-time composition: a polynomial algorithm that calls a polynomial subroutine polynomially many times runs in polynomial time.**

Which is what makes modular programming compatible with complexity analysis — you can reason about a subroutine's complexity in isolation and compose the results, which is not true for every resource bound (it fails for logarithmic space without care, Module 06 §1).

And it means polynomial-time reductions compose (Module 04 §1) — **without which NP-completeness would not be a transitive notion and Garey and Johnson's catalogue of 300 problems would not be usable**, since each new proof relies on chaining through earlier ones. **The closure property is load-bearing for the whole practice of the field.**

4 What membership in P does not tell you

- **Whether it is fast.** An $O(n^5)$ algorithm with a large constant is in P and is unusable on an input of a million elements — which is Module 01 §4's point, and it applies to every P-membership result.
- **Whether it parallelises.** NC is the parallel-tractable class, and whether $NC = P$ is open (Module 06 §4) — so a problem in P may be inherently sequential, which matters a great deal given CSCE 735's hardware.
- **Whether it fits in memory.** A polynomial-time algorithm may use polynomial *space*, which is an entirely separate and frequently binding constraint (Module 06).
- **Whether it streams, works online, or tolerates approximation** — all real engineering requirements that the class does not express at all.
- **And whether the problem instances you have are anything like the worst case** — which is Module 12, and is **frequently the only question that actually matters.** So 'it is in P' is good news and not an **engineering answer**: it tells you to look for an efficient algorithm rather than for a proof that none exists.

THE PRACTICAL READING OF P

'In P' means: a polynomial algorithm exists, so keep looking for a good one. The search is not futile, and that is the useful information — it licenses spending time.

'NP-hard' means: stop looking for an exact polynomial algorithm, and start on Module 11's approximations or Module 12's parameters — which is CSCE 627 Module 06 §4's redirection argument, with a weaker but still useful conclusion.

'Neither known' means both searches are live, and graph isomorphism is the standing example of a problem that stayed in that position for fifty years while serious people worked on both sides.

Which is the whole operational content of the classification. It redirects effort — and that is worth a great deal more than it sounds, because **the alternative is spending months on a search whose outcome was determined before you started.**

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Arora & Barak — chapter 1, and the discussion of the thesis (free draft)	P, the thesis, and the objections of §1 , stated carefully.
Edmonds — Paths, Trees, and Flowers (free)	§2's callout, in the original — and the introduction is where the thesis is argued. Worth reading for the argument rather than the algorithm.
Babai — Graph Isomorphism in Quasipolynomial Time (free)	§2's fourth row — the current state of the best-known NP-intermediate candidate.
Agrawal, Kayal & Saxena — PRIMES is in P (free)	§2's second row , and remarkably short for what it settled.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State the Cobham–Edmonds thesis** and write the strongest objection you can.
- 2 **Find the exponents of ten algorithms you know** and check §1's claim about natural problems.
- 3 **Construct a problem artificially in P with a large exponent** and note why it feels contrived.
- 4 **Verify that P is closed under polynomial composition**, with the exponent arithmetic.
- 5 **Show that P is closed under complement** and state why the same argument fails for NP.

- 6 **Look up the current best algorithm for graph isomorphism** and state its running time.
- 7 **Implement a Miller–Rabin primality test** and compare it against an AKS implementation if you can find one.
- 8 **Pick three problems from your own work** and determine which are in P.
- 9 **For one in P, find its best-known exponent** and assess whether that is practically adequate.
- 10 **Write down what 'in P' licenses you to do**, in one sentence.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Define P and state the Cobham–Edmonds thesis.
- 2 What kind of claim is the thesis?
- 3 Give five objections and the response to each.
- 4 What is Edmonds' argument, and why is it better?
- 5 Name four hard-won membership results and one still open.
- 6 What does graph isomorphism's status suggest about NP's structure?
- 7 Why does closure under composition matter so much?
- 8 Name five things membership in P does not tell you.
- 9 What does the classification operationally license?